WEEK 2: PYTHON FOR SECURITY AND HTTP FUNDAMENTALS REPORT

Project Title: Automated Web Application Security Assessment for Financial Platforms
Target Application: OWASP Juice Shop
Student: Robert Mayonde
Programme: Cybersecurity
Institution: JPR Solutions
Reporting Period: Week 2 of 12
Topic: Python for Security and HTTP Fundamentals

---

1. Introduction

The second week of the project focused on understanding Python programming for security scripting and the fundamentals of HTTP communication between web browsers and web applications.

HTTP is an important protocol in web application security because it defines how clients send requests to servers and receive responses. Understanding HTTP methods, headers, cookies and status codes is essential when assessing web application security.

Python was used to develop a basic HTTP interaction script using the Requests library. OWASP Juice Shop was selected as the target application because it provides a controlled environment for learning and practising web application security assessment.

This activity establishes a foundation for developing an automated security scanner capable of inspecting web applications and identifying potential security weaknesses.

2. Objectives

The objectives of Week 2 were:

1. To review Python fundamentals relevant to cybersecurity scripting.
2. To understand HTTP methods, particularly GET and POST.
3. To understand HTTP headers, cookies and status codes.
4. To develop a Python script that sends HTTP requests to a web application.
5. To inspect and document HTTP requests and responses during a login attempt.
6. To understand how HTTP analysis supports automated web application security assessment.

3. Tools and Technologies Used

The following tools and technologies were used or planned for this activity:

- **Python:** Used to write a basic HTTP interaction script.
- **Requests library:** Used to send HTTP requests and process responses.
- **OWASP Juice Shop:** Used as the local target web application.
- **Google Chrome Developer Tools:** Used to inspect browser network traffic.
- **Docker:** Used to run the target application locally.
- **Burp Suite Community Edition:** An optional tool for intercepting and analysing HTTP traffic.

4. Python HTTP Interaction Script

4.1 Purpose

A Python script was developed to communicate with the locally hosted OWASP Juice Shop application. The script sends an HTTP GET request and retrieves information from the server's response.

The purpose of this exercise was to understand how a security script can interact with a web application without relying entirely on a browser.

4.2 Implementation

The script uses the Requests library to send a GET request to the target application. It then displays the HTTP status code, content type, server header and response size. A short preview of the response body is also displayed.

The script includes error handling to manage situations such as connection failures and request timeouts.

4.3 Execution Results

Target URL: http://127.0.0.1:3000

Script filename: http_basics.py

Item	Observed result
Target URL	http://127.0.0.1:3000

HTTP status code	200
Content-Type	text/html; charset=UTF-8
Server header	None
Response size	9393 bytes
Execution result	Successful

```
PS C:\Users\Robert Mayonde\Downloads\Financial Platform Scanner> python src/http_basics.py
Target URL: http://127.0.0.1:3000
Status Code: 200
Content Type: text/html; charset=UTF-8
Server Header: None
Response Size: 9393 bytes

Response Preview:
<!--
  ~ Copyright (c) 2014-2026 Bjoern Kimminich & the OWASP Juice Shop contributors.
  ~ SPDX-License-Identifier: MIT
  -->

<!doctype html>
<html lang="en" data-beasties-container>
<head>
  <meta charset="utf-8">
  <title>OWASP Juice Shop</title>
  <meta name="description" content="Probably the m
PS C:\Users\Robert Mayonde\Downloads\Financial Platform Scanner>
```

Screenshot 1: the python script's results

4.4 Interpretation

The HTTP status code indicates how the server handled the request. The response headers provide additional information about the content and server response, while the response size indicates the amount of data received.

The response preview helps demonstrate that the script can retrieve content from a web application. This functionality provides a starting point for later scanner features, such as inspecting pages and checking response headers.

5. HTTP Fundamentals

5.1 HTTP Methods

HTTP methods describe the action requested by a client.

GET: Used to retrieve information from a server. In this activity, GET was used to request the Juice Shop home page.

POST: Commonly used to submit information to a server. Login forms and API authentication requests often use POST to transmit credentials or other authentication data.

Understanding these methods helps security analysts identify how a web application processes requests and handles user input.

5.2 HTTP Headers

HTTP headers provide additional information about requests and responses. Examples include Content-Type, User-Agent, Set-Cookie and Content-Security-Policy.

Security headers can help protect web applications against certain attacks. Their presence, configuration and relevance should be assessed when performing a security review.

5.3 HTTP Status Codes

Status codes indicate the result of an HTTP request. For example, 200 indicates success, 302 indicates a redirect, 401 indicates an authentication-related challenge or failure, 403 indicates forbidden access, and 404 indicates that a resource was not found.

These codes must be interpreted in context because a successful HTTP response does not necessarily mean an application is secure.

5.4 Cookies and Sessions

Cookies allow a web application to store information in the browser. Session cookies may help the application recognise an authenticated user across multiple requests.

Security attributes such as HttpOnly, Secure and SameSite help reduce certain cookie-related risks. Their configuration should be examined during the login traffic analysis.

5.5 HTTP and HTTPS

HTTP does not provide transport encryption by itself. HTTPS uses TLS to protect communication between a client and a server.

Financial applications should use properly configured HTTPS to protect credentials, session information and sensitive financial data during transmission.

The local HTTP environment used for this exercise is intended for controlled testing and should not be used to transmit real credentials.

6. HTTP Request and Response Analysis During Login

6.1 Methodology

Google Chrome Developer Tools was used to inspect the network traffic generated by OWASP Juice Shop during a login attempt.

The Network tab was used to identify the relevant request and examine its URL, method, headers, payload and response. The objective was to understand how the browser communicates with the application's server during authentication.

Only an authorised local test environment was used.

6.2 Login Request Details

The following was observed in Chrome developer tool;

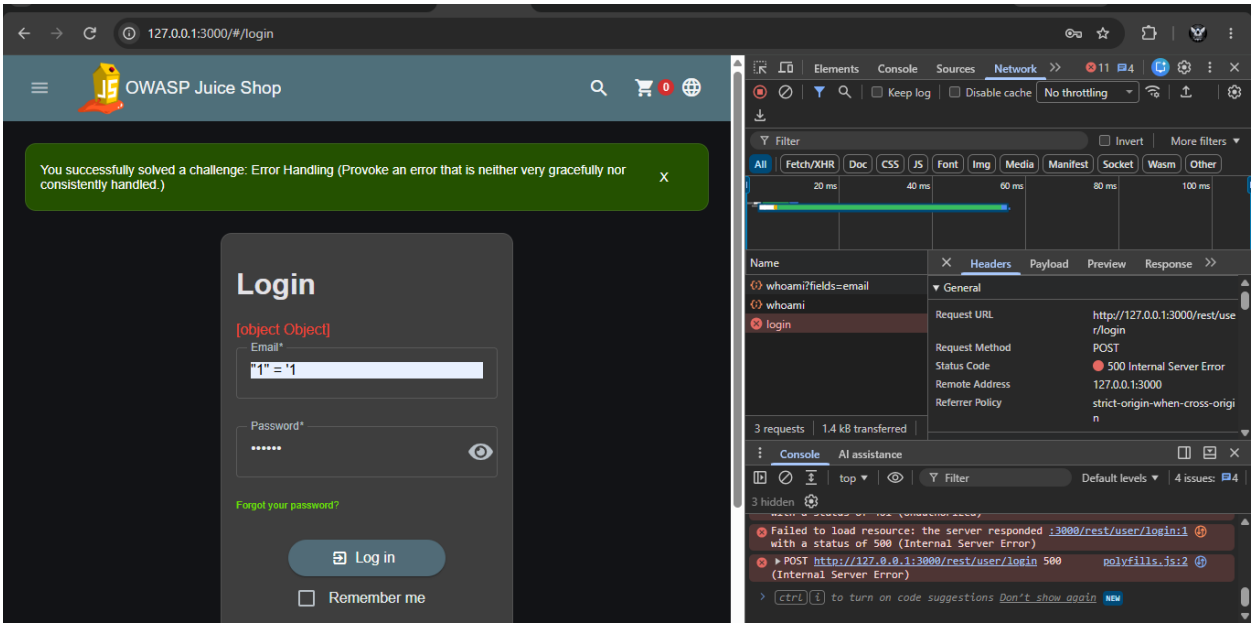
Field	Observed result
Request URL	http://127.0.0.1:3000/rest/admin/application-configuration
HTTP method	Get
Request Content-Type	application/json
Request payload structure	“1” = ‘1
Origin header	None
Authentication result	Unsuccessful

Screenshot 2: Insert a screenshot of the login request in Chrome Developer Tools, showing the request URL, method and relevant headers. Hide credentials and sensitive tokens.

6.3 Login Response Details

Field	Observed result
HTTP status code	500

Response Content-Type	application/json; charset=utf-8
Set-Cookie header	Present
Authentication outcome	Internal Server Error



Screenshot 3: Response in Chrome developer

6.4 Analysis and Interpretation

The login request and response demonstrate how a browser communicates with a web application's authentication endpoint.

The request method and payload help explain how authentication information is submitted. The response status and body indicate how the application handled the login attempt.

The analysis should also consider whether HTTPS is used, whether session cookies have appropriate security attributes, and whether relevant security headers are present.

The observations should be used to identify areas requiring further investigation rather than assuming that an individual missing header or unusual response automatically proves a vulnerability.

7. Security Observations

The following security considerations are relevant to the assessment:

1. **Transport security:** Authentication credentials and session data should be transmitted over HTTPS in production environments.
2. **Session management:** Session cookies should have appropriate security attributes, and session handling should be reviewed.
3. **Security headers:** Relevant response headers should be inspected to identify possible configuration weaknesses.
4. **Authentication responses:** Login errors should be reviewed for potentially excessive information disclosure.
5. **Request handling:** The application should validate input and enforce authentication and authorisation on protected resources.

These are assessment areas, not confirmed vulnerabilities. Any findings must be supported by evidence from the test environment.

8. Limitations

The Python script developed during this week performs basic HTTP interaction rather than a comprehensive vulnerability assessment.

The script's results are limited to the information it requests and displays. A successful page retrieval does not establish that the application is secure.

The login analysis is limited to the requests and responses observed during the selected test cases. Additional testing would be required to assess authentication controls, session management, access control and other potential security weaknesses.

9. Conclusion

Week 2 established a foundation for Python-based web application security assessment by introducing HTTP communication and basic request handling.

The Python script demonstrated how to send a request to a locally hosted web application and inspect the resulting response. The login traffic analysis provided a practical method for examining authentication-related requests, headers, status codes and responses.

These skills will support the continued development of the Automated Web Application Security Assessment for Financial Platforms project. Future work will build on this foundation by expanding the scanner's discovery capabilities, adding controlled security checks and improving the reporting of potential vulnerabilities.